智维Agent

SmartOpsAgent 技术文档

面向中小企业的可信 AI 智能运维与知识协同平台

AI Agent · RAG · MCP · 可信安全闭环

文档版本：v1.0（2026-10-25）

项目类型：AI Native Web 应用（前端 + AI 服务 + 知识服务 + Agent 框架 + 后端）

技术栈：Vue 3 · Vite · FastAPI · Spring Boot · Python Agent · 混合检索

目录

1. 项目概述
2. 需求分析
3. 系统架构设计
4. AI 技术方案（选型理由 · 集成方式 · 优化策略）
5. 核心功能实现
6. 测试与部署
7. 总结与展望
8. 创新点说明（技术创新点 · 应用创新点）

1. 项目概述

1.1 项目定位

智维Agent（SmartOpsAgent）是一个面向中小企业的**可信 AI 智能运维与知识协同平台**，基于 AI Agent + RAG + MCP + 可信安全闭环构建，是一套 AI Native Web 应用。它把"监测 → 分析 → 知识检索 → 证据验证 → 决策规划 → 风险判断 → 人工审批 → 安全执行 → 结果验证 → 审计留痕 → 经验沉淀"连成端到端可信闭环，让 AI 从"会回答运维问题"走向"可信地完成运维任务"。

1.2 要解决的核心问题

中小企业在日常运维中普遍面临：运维知识分散、故障定位依赖个人经验、传统检索返回错误版本知识、大模型结论不可验证、AI 能分析却无法安全执行、操作缺乏验证与审计。智维Agent 将上述环节在统一架构内贯通，使输出可解释、动作可控制、过程可追溯。

1.3 一句话价值定位

> 把大模型能力约束在"证据驱动、风险可控、全程留痕"的闭环里，让企业敢把生产环境交给一位"AI 运维同事"——结论有证据、动作有边界、过程有留痕。

1.4 技术栈概览

分层	技术选型
前端	Vue 3 + Vite 7 + TypeScript，共享逻辑层 TS（48 个模块）；Element Plus + Pinia + Vue Router + ECharts + Three.js
后端	Spring Boot 3.3.5 / Java 17，MyBatis，MySQL 8（119 张表），Redis 7；账户/权限/审计
AI 服务	FastAPI（:8001）：LLM 网关、对话编排、SSE 流式、RAG 检索与提示词组装
知识服务	FastAPI（:8002）：文档解析 → 分块 → 嵌入 → 混合检索 → 重排 → 证据与可信度评分 → 评测
Agent 框架	Python：planner / executor / inspector / memory / policies + kernel + 插件与工具生态
检索	混合检索（BM25 关键词 + 向量）+ 词法重排 + 版本感知过滤
工具集成	MCP 适配器、Function Calling、Docker/Kubernetes/MySQL/Nginx/Redis 插件
多模态	Three.js 3D + ECharts 图表 + 语音输入（Web Speech API）+ 截屏/拍照证据附件
模型接入	OpenAI 兼容网关（mock / Ollama / OpenAI 兼容，默认 mock 零密钥跑通）

2. 需求分析

2.1 背景与痛点

中小企业的 IT 运维普遍具备"人手少、系统杂、知识散"三个特征。具体痛点与现有方案不足如下：

现实痛点	现有方案的不足
运维知识散落在聊天记录、老员工经验、历史工单中	传统搜索引擎按关键词命中，返回的常是过期版本
故障定位依赖个人经验，新人上手周期长	通用问答机器人给不出可核对的证据链
大模型能"说"但不能"做"，或做了没人敢用	缺少风险分级、人工审批与审计留痕
一次误操作可能造成生产事故	通用 Agent 工具默认拥有过大的执行权限
故障处理过程无法复盘	操作缺少结构化记录，经验无法沉淀

2.2 目标用户与核心场景

目标用户：10–200 人规模企业的运维负责人、驻场运维工程师、技术负责人；以及高校实验室、实训机房等具备同类运维诉求的场景。

核心场景：

1. **故障问答与定位**：工程师用自然语言（可附截屏/拍照）描述故障，系统检索版本正确的知识并给出带证据的处置建议。
2. **证据驱动的知识检索**：同一主题存在多个版本时，按目标环境版本过滤，回答附来源与可信度评分。
3. **受控执行**：Agent 生成处置计划，策略引擎判定风险等级，高风险动作转人工审批后才执行。
4. **结果核查与经验沉淀**：执行后由 inspector 核查结果并写入审计，处置经验回流知识库。
5. **团队协同**：共享会话、团队工作区、资源授权与通知，支撑多人协作。

2.3 项目目标

功能目标

- 完成"监测→分析→检索→验证→规划→风险→审批→执行→核查→审计→沉淀"的可信闭环。
- 提供对话式统一入口，支持流式输出与 AI 思考过程可视化。
- 支持多模态输入：语音描述、截屏/拍照/图片证据。
- 提供用户层与管理层双入口，覆盖账户、权限、审计、系统治理。

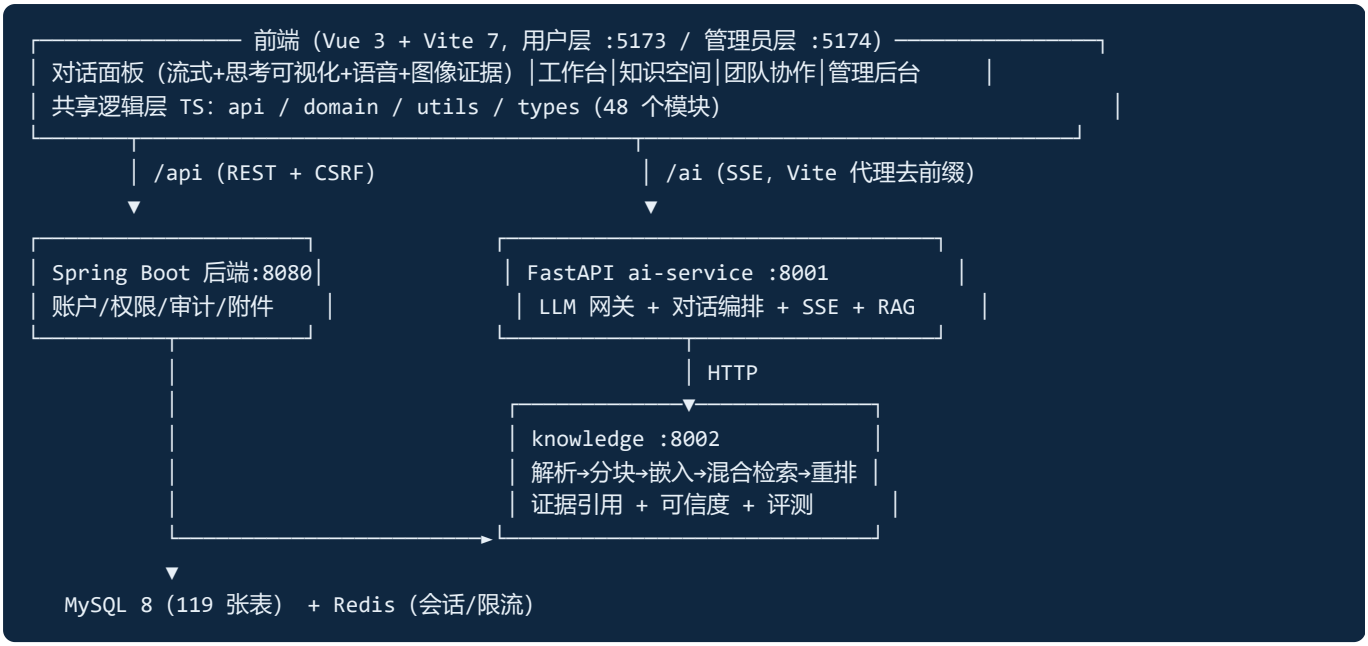
技术目标

目标	落地方式
AI 技术深度	Agent 五模块；混合检索 + 证据验证 + 可信度评分；SSE 流式与思考阶段事件
前端工程质量	Vue 3 + Vite 7；共享逻辑层 TS；构建分包；494 项前端测试 + 类型检查 + 双入口构建门禁
可运行性	默认 mock 模型零密钥跑通；一键切换 Ollama / OpenAI 兼容模型
安全可信	BCrypt(12)、人机校验、CSRF、MFA、风险策略引擎、人工审批、审计留痕

可扩展性	插件化工具（Docker/Kubernetes/MySQL/Nginx/Redis）、MCP 适配器、多会话存储后端

3. 系统架构设计

3.1 总体架构



3.2 分层设计

分层	职责与关键模块
前端（Vue 3）	对话/工作台/知识空间/团队协作/管理后台；共享逻辑层承载 api / domain / utils / types；ECharts 与 Three.js 负责可视化
后端（Spring Boot :8080）	业务数据、账户权限（RBAC）、附件会话、团队、审计；Spring Security + CSRF + 频率限制
AI 服务（FastAPI :8001）	LLM 网关、对话编排、SSE 流式、RAG 检索与提示词组装
知识服务（FastAPI :8002）	文档解析、分块、嵌入、混合检索、词法重排、证据与可信度评分、回归评测
Agent 框架（Python）	planner / executor / inspector / memory / policies + kernel + 插件与工具（MCP）
存储	MySQL 8（119 表）持久化业务与审计；Redis 承载会话与限流

3.3 可信闭环数据流

```
用户提问（文本 / 语音 / 图像文件名）
  → 前端 SSE 客户端（解析 thinking / token / done）
  → ai-service：RAG 检索 → 证据验证 → Prompt 组装（系统提示 + 知识证据 + 图像证据段）
  → LLM 网关（mock / Ollama / OpenAI 兼容）
  → SSE 回流：thinking（vision→retrieve→evidence→plan→risk→answer）→ token → done（引用 + trust_score）
  → 高风险动作：策略引擎 → 人工审批 → 受控执行 → 结果核查 → 审计留痕
```

3.4 安全与权限设计

- **认证与防护**：BCrypt(12) 密码哈希、会话/CSRF/频率限制、管理员 MFA、人机校验策略。
- **风险策略引擎**：`agent/policies/policy_engine.py` 依据权限与风险注册表给每个动作打风险等级。
- **人工审批**：高风险动作进入 `agent/safety/approval.py` 审批队列，未获批不执行。
- **受控执行与审计**：`agent/executor/audit.py` 在沙箱与权限约束下执行，产出结构化结果与审计记录。
- **密钥治理**：模型密钥 AES-GCM 加密存储；所有密钥经环境变量注入，绝不入库。

4. AI 技术方案

本节重点阐述 AI 技术的**选型理由**、**集成方式**与**优化策略**。核心设计原则贯穿全链路：`证据先行` `风险分级` `全程留痕` 。

4.1 AI 技术选型与理由

能力	选型	选型理由
大语言模型接入	OpenAI 兼容网关（mock / Ollama / OpenAI 兼容）	解耦模型厂商，运行期可切换；默认 mock 零密钥即可完整演示，企业落地可无缝切到自有模型
知识检索（RAG）	混合检索：BM25 关键词 + 向量召回	单一路径易漏召回——关键词保高召回率与版本可解释，向量补语义相似，二者并行更稳
版本治理	`version_filter` 版本感知过滤	中小企业最常见故障是"查到旧版本"，把版本作为检索一等公民是区别于通用 RAG 的实质差异
可信表达	Citation + `trust_score` 可信度评分	让结论可验证、可解释，引用带来源/分块/版本/分数，避免"黑盒断言"
Agent 框架	planner/executor/inspector/memory/policies	把"权限、风险、审批、核查、审计"做成可运行代码路径，而非单点 AI 调用
推理透明化	SSE 六步思考事件	用户能"看见"AI 在哪一步检索了什么、命中多少证据，提升对 AI 的信任
工具集成	MCP 适配器 + Function Calling + 插件契约	把运维能力从硬编码变为可插拔资产，新环境只需增加插件
多模态	隐私边界分层（语音/图像/附件）	在演示多模态的同时划清隐私红线，符合合规要求

4.2 Agent 框架（五模块）

- **planner**：把故障描述拆解成有依赖关系的步骤计划（`agent/planner/steps.py`）。
- **executor**：按步骤调用工具，产出结构化结果与审计记录。
- **inspector**：规则判定 + 证据判定 + AI 判定四级证据核查（规则→证据→AI→状态核验），输出验证结论（`agent/inspector/judges/`）。
- **memory**：短期上下文与长期经验（`operator_memory`）双层记忆，越用越懂你。
- **policies**：风险等级与权限策略，高危动作默认拒绝或转审批。
- **kernel**：插件注册、依赖解析、事件总线、生命周期与运行时加载。

4.3 混合检索与证据可信度（RAG）

1. **召回层**：BM25 关键词召回与向量召回并行（`knowledge/retrieval/hybrid.py`），避免单一路径漏召回。
2. **过滤层**：`version_filter` 按目标环境版本过滤候选，解决"知识正确但版本不对"的行业通病。
3. **收敛层**：词法重排器对候选重新排序（`knowledge/reranker/lexical_reranker.py`）。

4. **表达层**：检索结果转为 Citation（来源 / chunk / 版本 / 分数），聚合为 `trust_score`（`knowledge/evidence/`）。
5. **评测层**：`knowledge/evaluation` 内置指标与样例数据集，支持回归评估。

4.4 可切换模型网关

- 默认 `mock` 提供方，无需任何密钥即可完整演示全链路。
- 支持 Ollama 本地模型与 OpenAI 兼容接口，运行期切换（`ai-service/app/services/model_gateway.py`）。
- 后端维护模型连接、路由、密钥（AES-GCM 加密存储）、Token 预算与用量统计。

4.5 集成方式

集成点	方式
前后端	REST（`/api`，CSRF）+ SSE（`/ai`，Vite 代理去前缀）；前端 `src/api/stream.ts` 逐事件渲染
AI 与知识	ai-service 通过 HTTP 调用 knowledge 的检索/证据/可信度接口，组装进 Prompt
工具生态	Function Calling + MCP 适配器（`agent/tools/adapters/mcp.py`）+ `AgentPlugin` 契约（`agent/sdk/plugin.py`）
会话存储	Redis / MySQL / PostgreSQL / SQLite / 文件五种后端（`agent/plugins/session/`）
多模态	语音（Web Speech API 浏览器内识别）、图像（仅文件名参与链路）、附件（私有会话隔离）

4.6 优化策略

- **检索优化**：词法重排收敛候选、版本过滤去噪；向量嵌入默认用轻量 `hash_embedding.py`，可切 Sentence Transformers（`sentence_transformer.py`）提升语义质量。
- **提示词优化**：Prompt 模板集中于 `agent/prompts/`，按"系统提示 + 知识证据 + 图像证据段"组装，输出稳定可控；结合 few-shot 提升一致性。
- **性能与分包**：统一分包策略，`vendor-echarts` 由 510 kB 降至 346 kB（zrender 独立拆分），Three.js 保持懒加载；共享 TS 逻辑层减少重复构建。
- **分流与降级**：默认 mock 提供方保证零密钥可演示；模型路由 + Token 预算做成本与限流控制。
- **隐私优化**：音频不出端、图片内容不出浏览器、附件与模型分析隔离，从源头降低合规风险。
- **风险优化**：策略引擎风险分级、高危转人工审批、沙箱执行，把"可信"落到代码路径而非文档。

5. 核心功能实现

5.1 用户层功能

模块	主要能力
AI 对话	流式问答、思考阶段可视化、引用证据与可信度评分、语音输入、图像证据、附件会话
运维工作台	服务器与指标监测、会话与命令记录、运行证据与输出留存
知识空间	个人知识库、候选知识审阅、版本感知检索
团队协作	团队工作区、成员与能力、Agent 运行与审批、草稿版本、通知
系统可视化	ECharts 图表、Three.js 3D 拓扑

5.2 管理员层功能

模块	主要能力
账户与权限	用户/角色/部门/岗位、RBAC、会话管理、密码策略
安全治理	管理员 MFA、人机校验策略、登录日志、审计查询
模型网关	模型连接、路由、密钥托管（AES-GCM）、Token 预算与用量
平台治理	字典、菜单、公告、系统监控

5.3 服务与框架实现

- **ai-service（:8001）**：LLM 网关、对话编排、SSE 流式、RAG 检索与提示词组装；`chat_stream` 输出 thinking / token / done 三类事件。
- **knowledge（:8002）**：文档解析、分块、嵌入、混合检索、重排、证据与可信度评分、评测一体化。
- **agent（Python）**：五模块闭环 + 插件与工具生态，覆盖规划到审计全链路。
- **backend（:8080）**：业务数据、账户权限、附件会话、团队与审计，MySQL 8 + Redis 存储。

5.4 多模态与隐私边界实现

输入类型	处理方式	数据是否出端
语音	Web Speech API 在浏览器内识别为文本，仅填入输入框（`domain/chat/speechInput.ts`）；不支持时入口自动隐藏	音频不出端
图像证据	本地暂存，随消息仅传**文件名列表**给 ai-service，触发"图像证据解析"阶段	图片内容不出端
文本/日志附件	走私有附件会话（`chat_conversation` / `chat_attachment`），仅保存证据，不参与模型分析	由用户显式上传，与模型分析隔离

6. 测试与部署

6.1 测试策略

- **前端质量门禁**：494 项前端测试覆盖契约、安全不变量与领域逻辑；`tsc --noEmit` 类型检查纳入门禁；双入口（用户层/管理层）构建门禁保证同源。
- **分层测试**：单元测试、集成测试、E2E 测试覆盖 Agent 与 API；`pytest.ini` 统一编排。
- **算法评测**：`docs/experiments/` 提供评测计划、消融实验、检索评测、可信度评测，配合 OpsBench 样例数据集进行回归。
- **可信度验证**：对检索召回、证据命中、`trust_score` 分布做量化评估，确保"可解释"可被度量。

6.2 部署架构

- **容器化**：`deploy/` 提供 docker-compose、MySQL 初始化 SQL，一键拉起多服务。
- **存储**：MySQL 8（业务与审计，119 张表）+ Redis 7（会话/限流）。
- **可观测性**：`infrastructure/` 集成 Prometheus / Grafana / Loki / Promtail，覆盖指标、日志与告警。
- **跨平台**：Windows 提供 `scripts/windows/*.ps1`，Linux/macOS 提供等效 shell 脚本；`.env.example` 模板化所有密钥，密钥一律留空不入库。

6.3 运行与可复现

> 默认 `LLM_PROVIDER=mock`，无需任何模型密钥即可完整跑通"提问 → 检索 → 规划 → 审批 → 执行 → 核查 → 审计"全链路；评委与用户可零配置复现，企业落地时再切到 Ollama / OpenAI 兼容模型。

启动顺序：AI 服务（:8001）→ 知识服务（:8002）→ 后端（:8080，Spring Boot）→ 前端（:5173 用户层 / :5174 管理员层）。首次管理员由 `SMARTOPS_BOOTSTRAP_USERNAME/PASSWORD` 引导创建（仅账户表为空时生效）。

7. 总结与展望

7.1 成果总结

智维Agent 把"规划—风险—执行—验证—审计"做成端到端可信闭环，而非单点 AI 调用；以版本感知的混合检索 + 证据可信度评分解决中小企业"查到旧版本"的普遍痛点；以 SSE 六步思考事件把"黑盒回答"变成"透明推理"。系统在默认 mock 零密钥下即可完整演示，并支持切换自有模型，兼顾可演示与可生产。

7.2 局限与诚实声明

- 图像当前为**文件名级**参与演示链路，尚未做图像内容级理解；路线图为接入 OCR（`knowledge/ocr/`）与多模态大模型，并在用户授权后上传。
- 语音仅做浏览器内识别，不涉及声纹与音频留存。
- 混合检索的向量嵌入默认使用轻量哈希嵌入，可切换 Sentence Transformers 提升语义质量。

7.3 后续演进

- 端侧推理（Transformers.js / WebGPU），降低推理成本与延迟。
- 多节点会话与 MFA 强化、团队协同深化。
- 图像内容级理解与多模态大模型接入（授权后上传）。
- SaaS / 私有化双形态交付，扩大中小企业覆盖。

8. 创新点说明

本节明确区分**技术创新点**与**应用创新点**，每个创新点均对应可运行代码与可演示路径，非概念包装。

8.1 技术创新点

① 可解释的"可信 AI 闭环"

把运维任务拆成"规划—审批—执行—核查—沉淀"五个可审计阶段，由 Agent 五模块分别承担（`planner/steps.py`、`policies/policy_engine.py`、`safety/approval.py`、`executor/audit.py`、`inspector/judges/`）。多数作品停留在对话层，本项目把权限、风险、审批、核查、审计做成可运行代码路径。

② 版本感知的混合检索 + 证据可信度评分

BM25 + 向量并行召回（`hybrid.py`），`version_filter` 按环境版本过滤，词法重排收敛，结果转为 Citation 并聚合 `trust_score`。把"版本"作为检索一等公民，是与通用 RAG 的实质差异。

③ 把"黑盒回答"变成"透明推理"

`chat_stream` 以 SSE 输出 thinking / token / done 三类事件；思考阶段固定六步：vision→retrieve→evidence→plan→risk→answer。前端逐事件点亮，用户可见 AI 在每一步检索了什么、命中多少证据、判定何种风险。

④ 插件化 Agent 工具生态

工具通过 `AgentPlugin` 契约（`agent/sdk/plugin.py`）注册，kernel 完成注册/依赖解析/生命周期/事件分发；内置 Docker/Kubernetes/MySQL/Nginx/Redis 五类插件；MCP 适配器可接入外部工具生态。运维能力从硬编码变为可插拔资产。

⑤ 可切换模型网关 + 零密钥可演示

默认 mock 提供方零密钥演示全链路；支持 Ollama 与 OpenAI 兼容接口运行期切换；后端 AES-GCM 加密托管密钥并做 Token 预算。兼顾"可演示"与"可生产"。

⑥ 前端工程化创新（双入口同源架构）

用户层/管理层共用代理工厂 `vite.shared.mjs`，避免两端口各写代理导致会话与 CSRF 分裂；共享逻辑层 TS（48 模块）承载 api/domain/utils/types，`tsc --noEmit` 纳入门禁；494 项前端测试守护契约与安全不变量。

8.2 应用创新点

① 选题创新：聚焦中小企业"可信运维"

不追"更大的模型"，而切入"人手少、系统杂、知识散"的中小企业真实且未被充分满足的运维可信需求，把 AI 定位为"可托付的运维同事"。

② 多模态输入的隐私边界分层

行业常见"多模态 = 全部上传"，本项目把"能用"和"该传"分开：语音仅浏览器内识别（音频不出端）、图像仅传文件名（内容不出端）、附件与模型分析隔离，并用 UI 文案提示用户遮盖密码与密钥。

③ 交互创新：降低运维 AI 使用门槛

AI 思考过程可视化 + 流式输出 + 语音输入，让非专家也能看懂 AI 的推理与证据，直接提升对 AI 的信任与采用率；不支持语音时入口自动隐藏（渐进增强）。

④ 可落地创新：演示即生产路径

默认 mock 零密钥可复现、模型可一键切换，使同一套代码既可用于竞赛演示，也可平滑转入企业私有化/自有模型生产环境，缩短从原型到落地的距离。

⑤ 治理创新：运维治理闭环

用户层与管理层双入口覆盖账户、权限、审计、系统治理，配合团队工作区与审批流，形成"人 + AI + 流程"协同的运维治理体系，支撑 SaaS / 私有化双形态。

> **可验证证据对照**：可信闭环 → 管理端审批与审计页面；版本感知检索 → 同一主题多版本检索对比；透明推理 → 对话面板思考阶段逐条点亮；多模态边界 → 语音输入 + 图像证据 + 附件隔离；插件生态 → 插件列表与工具调用；模型网关 → mock/Ollama 切换；前端工程化 → `npm test` 494 项通过。

文档基于 SmartOpsAgent 仓库（README.md、docs/SMARTOPS_ARCHITECTURE.md、docs/architecture/overall.md、docs/competition/innovation.md、docs/competition/project-plan.md）整理生成，内容均对应仓库内真实模块与路径。许可证：MIT。